Оператор циклу `for` та функція `range()`

7

Оператори циклів використовують у випадках, коли в програмі необхідно багаторазово виконати певну послідовність команд. У мові Python основним циклом із заздалегідь відомою кількістю повторень є цикл `for`.

```
Синтаксис циклу for
for параметр in range(start, stop, step):
    блок_операторів
```

1. Генератор послідовностей `range()`

Найчастіше для формування послідовностей чисел у циклі `for` використовується вбудована функція `range()`. Вона може приймати від 1 до 3 аргументів:

- ☑ **`range(stop)` "— генерує числа від** 0 **до** $stop - 1$ **із кроком** 1**.**
- ☑ **`range(start, stop)` "— генерує числа від** $start$ **до** $stop - 1$ **із кроком** 1**.**
- ☑ **`range(start, stop, step)` "— генерує числа від** $start$ **до** $stop - 1$ **із кроком** $step$**.**

Виклик `range()`	**Згенерована послідовність**	**Примітка**
`range(5)`	`0, 1, 2, 3, 4`	Від 0 до 4
`range(1, 6)`	`1, 2, 3, 4, 5`	Від 1 до 5
`range(2, 10, 2)`	`2, 4, 6, 8`	Парні числа від 2 до 8
`range(10, 0, -1)`	`10, 9, 8, 7, 6, 5, 4, 3, 2, 1`	Зворотний відлік

Table 7.1: Приклади роботи функції `range()`

Якщо передати в range() лише одне число, за замовчуванням відлік починається з 0, а верхня межа не включається

```
for i in range(5):
    print(i)
```

Результат виведення : 0, 1, 2, 3, 4

range(start, stop) починає з числа start (включно) і зупиняється перед stop (не включаючи його).

```
for i in range(1,5):
    print(i)
```

Результат виведення: 1, 2, 3, 4

Третій параметр — це крок (step). range(1, 10, 2) починає з 1 і додає 2 на кожному кроці, генеруючи непарні числа до 10

```
for i in range(1,10,2):
    print(i)
```

Результат виведення: 1, 3, 5, 7, 9

Якщо крок від'ємний (-1), значення будуть зменшуватися. Цикл починається з 10 і зупиняється перед 0.

```
^^I
for i in range(10,0,-1):
    print(i)
```

Результат виведення: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

2. Базові алгоритми з циклом `for`

Приклад 1. Обчислення суми натуральних чисел від 1 до n

```
n = int(input("Введіть n "))
s = 0
for i in range(1, n + 1):
    s += i

print(f"Сума чисел від 1 до {n} дорівнює: {s}")
```

input(...) запитує у користувача число та приймає його як рядок (текст).

int(...) перетворює введений текст у ціле число та зберігає його в змінну n.

$s = 0$ створюється змінна-накопичувач s із початковим значенням 0. У ній буде зберігатися підсумкова сума.

for i in range(1, n + 1): цикл for перебирає числа по порядку.

range(1, n + 1) генерує послідовність від 1 до n включно (верхня межа $n + 1$ не включається в діапазон, тому цикл зупиняється саме на n).

s += i на кожній ітерації циклу поточне число i додається до суми s (це скорочений запис для $s = s + i$).

print(f"Сума чисел від 1 до n дорівнює: s")

За допомогою f-рядка на екран виводиться підсумковий результат.

Приклад 2. Обчислення факторіала n!

```
n = int(input("Введіть n "))
fact = 1

for i in range(1, n + 1):
    fact *= i

print(f"{n}! = {fact}")
```

Приклад 3. Табулювання функції y = sin(x) на відрізку [a, b]

```
import math

a = float(input("Введіть початок відрізка a "))
b = float(input("Введіть кінець відрізка b "))
n = 10
h = (b - a) / n

print(" x | sin(x)")
print("-------------")
for i in range(n + 1):
    x = a + i * h
    y = math.sin(x)
    print(f"{x:6.2f} | {y:6.4f}")
```

Форматоване виведення (виділений рядок):

$\{x : 6.2f\}$ — виводить значення x: занімає 6 символів у ширину та округлюється до 2 знаків після коми.| — виводить розділювальну риску між стовпчиками.

$\{y : 6.4f\}$ **— виводить значення** y $(\sin(x))$**: шириною в 6 символів та округлене до 4 знаків після коми.**

3. Аналіз дільників числа та перевірка на простоту

Математична довідка

Просте число ”— це натуральне число, більше за 1, яке має точно два різні натуральні дільники: 1 та саме це число. Якщо число n **має дільник, відмінний від 1 та** n**, то найменший такий дільник не перевищує** $\lfloor\sqrt{n}\rfloor$**.**

Приклад 4. Виведення всіх дільників числа

```
n = int(input("Введіть натуральне число n "))
count=0
for i in range(1, n + 1):
    if n % i == 0:
        print(i)
    count=count+1
print(f"Кількість дільників: {count}")
```

Пошук дільників: Програма перебирає всі цілі числа i **від** 1 **до** n **включно і перевіряє умову n**

Виведення та лічильник: Якщо число i **є дільником:воно одразу виводиться на екран (print(i)); лічильник count збільшується на** 1**. Наприкінці програма виводить загальну кількість знайдених дільників.**

4. Обчислення нескінченних математичних рядів

4.1. Сума геометричного ряду

Обчислимо суму $S = \sum_{n=0}^{N} x^n = 1 + x + x^2 + \cdots + x^N$ **(**$|x| < 1$**).**

Приклад 5. Сума геометричного ряду

```
x = float(input("Введіть x (|x| < 1) "))
n = int(input("Введіть кількість членів N "))

s = 0.0
term = 1.0

for i in range(n):
    s += term
    term *= x

print(f"Сума ряду: {s:.6f}")
print(f"Точна сума 1/(1-x): {1 / (1 - x):.6f}")
```

4.2. Обчислення експоненти e^x через ряд Тейлора

Приклад 6. Обчислення e^x

```
import math

x = float(input("Введіть x "))
n = int(input("Кількість членів ряду "))

s = 0.0
term = 1.0

for i in range(1, n + 1):
    s += term
    term *= x / i

print(f"Наближене значення e^{x} = {s:.8f}")
print(f"Точне значення (math.exp): {math.exp(x):.8f}")
```

f"..." (f-рядок) — дозволяє вставляти значення змінних безпосередньо всередину тексту за допомогою фігурних дужок .

$\{s:.8f\}$ — бере значення змінної s (суму ряду) та форматує його як число з плаваючою крапкою, залишаючи рівно 8 знаків після коми.

Цей рядок показує наближене значення експоненти e^x , вираховане через суму перших n членів ряду Тейлора.

5. Управління циклом: `break`, `continue` та `else`

- ☑ **`break` ”— достроково припиняє виконання циклу.**
- ☑ **`continue` ”— пропускає залишок тіла циклу та переходить до наступної ітерації.**
- ☑ **`else` ”— блок коду, який виконується після завершення циклу, якщо вихід з нього не відбувся за допомогою `break`.**

Приклад 7. Пошук першого від’ємного числа з використанням break та else

```
for i in range(5):
    val = int(input("Введіть число "))
    if val < 0:
        print(f"Знайдено перше від'ємне число: {val}")
        break
else:
    print("Усі введені числа є невід'ємними.")
```

6. Оператори інкременту та особливості `float`

У Python відсутні оператори `++` та `--`. Замість них використовуються комбіновані оператори присвоєння:

- ☑ **`x += 1` замість `x = x + 1`**
- ☑ **`x -= 1` замість `x = x - 1`**
- ☑ **`x *= 2` замість `x = x * 2`**

Обережно з дійсними числами (float)!

Числа з плаваючою крапкою зберігаються в пам'яті комп'ютера у двійковій системі з обмеженою точністю (53 біти). Через це обчислення можуть містити незначну похибку:

Особливості float

```
>>> 0.1 + 0.1 + 0.1
0.30000000000000004
```

Для порівняння речових чисел завжди використовуйте функцію `math.isclose(a, b)` або порівняння з точністю ε: `abs(a - b) < 1e-9`.